

Modeling Spotify’s Shuffle Algorithm in Python

A practical reconstruction of the publicly described “Fewer Repeats” approach

Purpose. This document explains what Spotify publicly says about Shuffle, translates that description into an implementable Python design, and shows how the Spotify Web API can be used to read playlist data or toggle playback shuffle. The Python scoring model below is illustrative; Spotify does not publish its complete production code, exact candidate count, weighting constants, or all ranking signals.

Core idea: generate several truly random playlist orders, penalize orders that place recently heard tracks too early, and choose the candidate with the best freshness score. This preserves randomness while reducing the feeling of repetition.

1. What Spotify has publicly described

Spotify Engineering described a system called “Fewer Repeats” in November 2025. At a high level, Spotify says it (1) generates multiple mathematically random candidate sequences, (2) scores each sequence for freshness based on how recently tracks were heard, and (3) chooses the freshest candidate. Tracks heard more recently receive a larger penalty when they appear near the front of a candidate order.

- The starting point remains random: each candidate is a random permutation of the playlist.
- Freshness is evaluated after randomization rather than by deterministically sorting tracks by recency.
- The public description references listening recency within the playlist and across Spotify.
- Spotify has not published the complete production formula or tuning constants, so any code recreation must choose its own parameters.

Important API distinction: the Spotify Web API exposes a playback control that turns shuffle on or off, but it does not return or let a developer configure Spotify’s internal shuffle-ranking formula.

2. Proposed Python architecture

A clean implementation can be split into four components:

Component	Responsibility	Example
Playlist loader	Provide the available tracks and stable IDs.	Spotify API, JSON fixture, or Python list
History provider	Map track IDs to the last time each track was heard.	Recently played data or local history store
Candidate generator	Create N unbiased random permutations.	<code>random.sample(...)</code> or <code>random.shuffle(...)</code>
Freshness scorer	Penalize recent tracks more strongly near the front.	Weighted recency penalty

3. Freshness scoring model

One practical scoring model is: for every track in a candidate order, compute a recency penalty and multiply it by a position weight. Early positions receive a larger weight. The total candidate score is the sum of those penalties; lower is better.

Illustrative formula

$$\text{score}(\text{sequence}) = \sum \text{position_weight}(i) \times \text{recency_penalty}(\text{track}_i)$$

- Recency penalty can decay exponentially as the time since last playback increases.
- Position weight can decay as the track moves deeper into the queue.
- Never-heard tracks can receive a penalty of zero, making them “fresh.”
- A seeded random-number generator can make tests reproducible while production runs remain unseeded.

4. Standalone Python implementation

The following example does not require Spotify credentials. It models the public “multiple random candidates + freshness scoring” concept with explicit, tunable parameters.

```
from __future__ import annotations

import math
import random
from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Iterable

@dataclass(frozen=True)
class Track:
    id: str
    name: str
    last_played: datetime | None = None

def recency_penalty(track: Track, now: datetime, half_life_hours: float = 72.0) -> float:
    """Return 0..1. Recently heard tracks approach 1; old tracks approach 0."""
    if track.last_played is None:
        return 0.0

    age_hours = max((now - track.last_played).total_seconds() / 3600.0, 0.0)
    return math.exp(-math.log(2) * age_hours / half_life_hours)

def position_weight(index: int, decay: float = 0.18) -> float:
    """Make a repetition near the start more costly than one much later."""
    return math.exp(-decay * index)

def freshness_score(sequence: list[Track], now: datetime) -> float:
    return sum(
        position_weight(i) * recency_penalty(track, now)
        for i, track in enumerate(sequence)
    )

def fewer_repeats_shuffle(
    tracks: Iterable[Track],
    candidates: int = 32,
    seed: int | None = None,
) -> list[Track]:
    """Choose the freshest of several independently randomized permutations."""
    pool = list(tracks)
```

```

if len(pool) < 2:
    return pool

rng = random.Random(seed)
now = datetime.now(timezone.utc)

best_order: list[Track] | None = None
best_score = float("inf")

for _ in range(candidates):
    candidate = rng.sample(pool, k=len(pool))
    score = freshness_score(candidate, now)
    if score < best_score:
        best_order = candidate
        best_score = score

return best_order or pool

```

5. Example usage and testability

```

from datetime import datetime, timedelta, timezone

now = datetime.now(timezone.utc)
tracks = [
    Track("t1", "Song A", now - timedelta(minutes=20)),
    Track("t2", "Song B", now - timedelta(hours=3)),
    Track("t3", "Song C", now - timedelta(days=2)),
    Track("t4", "Song D", now - timedelta(days=14)),
    Track("t5", "Song E", None),
]

order = fewer_repeats_shuffle(tracks, candidates=50, seed=7)
for position, track in enumerate(order, start=1):
    print(position, track.name)

```

Why a seed matters in tests. Randomized systems can be difficult to test. Passing a fixed seed makes a test run deterministic, while omitting the seed gives normal run-to-run variation.

6. Integrating with the Spotify Web API

Spotify authorization uses OAuth 2.0 and scopes. For playlist access, the exact scope depends on playlist visibility and ownership. Spotify's current Get Playlist Items endpoint is documented for playlists owned by the current user or playlists on which the user is a collaborator. For playback shuffle, the documented endpoint is PUT /me/player/shuffle and requires the user-modify-playback-state scope; Spotify documents that playback control as Premium-only.

```

import requests

API = "https://api.spotify.com/v1"

def set_spotify_shuffle(access_token: str, enabled: bool, device_id: str | None = None) -> None:
    params = {"state": str(enabled).lower()}
    if device_id:
        params["device_id"] = device_id

    response = requests.put(
        f"{API}/me/player/shuffle",
        headers={"Authorization": f"Bearer {access_token}"},
        params=params,
        timeout=15,
    )
    response.raise_for_status()

```

```
# Example:
# set_spotify_shuffle(token, True)
```

What this API call does — and does not do. It asks Spotify to enable or disable the service’s own shuffle behavior on a playback device. It does not upload the custom Python order above. If you need a visible custom order, a separate design is to create or modify a playlist using playlist-write endpoints, subject to Spotify’s current developer policies and user authorization.

7. Production design considerations

Concern	Recommendation
Candidate count	More candidates increase the chance of finding a fresh-feeling ordering but add CPU work. Benchmark rather than assuming 32 or 50 is optimal.
Recency horizon	A 72-hour half-life is only an example. A longer half-life suppresses recently heard tracks for longer.
Fairness	Do not make freshness penalties so strong that some tracks are effectively starved.
Artist clustering	If desired, add a separate penalty when the same artist appears within a short window. This is an extension, not part of the minimal public description above.
History gaps	Treat missing listening history explicitly. “Never heard” and “unknown” may need to be separate states.
Observability	Log aggregate scoring metrics, not sensitive user data. Measure repeat rate, skip rate, and distribution by position.
Rate limits and API change	Keep API access behind a small adapter so endpoint or policy changes do not affect the shuffle logic.

8. Automated workflow

A reusable automation can follow this pipeline:

1. Request user authorization and obtain an access token with only the scopes the application needs.
2. Load playlist items and normalize them into Track objects.
3. Load or construct a last-played map from an allowed history source.
4. Generate multiple random candidate permutations.
5. Score every candidate for freshness.
6. Select the lowest-scoring candidate.
7. Use the result locally for analysis/testing, or write a permitted reordered playlist if that is the product goal.
8. If the goal is simply Spotify playback shuffle, call the documented shuffle toggle endpoint instead of trying to replace Spotify’s internal algorithm.

9. Limitations and accuracy notes

- This is a reconstruction of Spotify’s public engineering description, not a reverse-engineered copy of proprietary production code.
- The candidate count, recency half-life, position-decay factor, and any artist-level constraints in the sample are implementation choices for demonstration.

- Spotify API availability, access rules, and endpoint behavior can change; verify the current developer documentation before deploying.
- Do not download Spotify audio or use Spotify content in ways prohibited by Spotify Platform policy.

References

- Spotify Engineering — “Shuffle: Making Random Feel More Human” (Nov. 2025) — <https://engineering.atspotify.com/2025/11/shuffle-making-random-feel-more-human>
- Spotify Web API — Toggle Playback Shuffle — <https://developer.spotify.com/documentation/web-api/reference/toggle-shuffle-for-users-playback>
- Spotify Web API — Get Playlist Items — <https://developer.spotify.com/documentation/web-api/reference/get-playlists-items>
- Spotify Web API — Authorization — <https://developer.spotify.com/documentation/web-api/concepts/authorization>
- Spotify Web API — Scopes — <https://developer.spotify.com/documentation/web-api/concepts/scopes>
- Spotify Web API — Create Playlist — <https://developer.spotify.com/documentation/web-api/reference/create-playlist>

Document status: technical implementation guide based on public Spotify documentation available in September 2026.